

Imagized Language Modeling with Factorized Glyph Embeddings and Diffusion-Based Generation

An Unsupervised Multilingual Framework

Lachlan Chen AgInTiFlow
AgInTi Lab, LazyingArt LLC

May 31, 2026

Abstract

We propose and analyze an **Imagized Language Model (ILM)** that represents text as images and employs a diffusion-inspired generation process. The ILM is built on four components: (1) *Glyph encoding* of text into images, (2) a *differentiable product code* embedding layer that factorizes lexical representations into multiple discrete “color” channels, (3) *contrastive alignment* of these discrete codes using question–answer pairs (or pseudo-pairs) with a novel second-order (Gram matrix) consistency loss to induce semantic structure, and (4) a *diffusion-based infiller* that treats sequences of discrete codes as image-like grids and learns to reconstruct masked parts. We derive the training objectives for each stage and explain why the combined optimization leads to emergent semantic organization of the code space. In particular, we show how the factorized codebooks capture latent aspects of meaning (e.g. syntactic role, semantic category, tone) through information-theoretic pressure and how the contrastive losses drive the model to “understand” words by their usage. The framework is language-agnostic and can be applied to scripts with no known translation or tokenization (e.g. historical scripts)—the model learns to interpret such languages purely from unannotated corpora by discovering a structured internal lexicon. We provide theoretical analysis and literature connections, outlining how this approach enables *learning meaning just by reading* in any language.

1 Introduction

Natural language is traditionally modeled as sequences of discrete tokens (characters, bytes, or words) mapped to continuous vectors. In this work, we depart from that view by representing language in a *visual modality*: each token is rendered as a small image (*glyph*), and sequences of tokens form a 2D image grid. This *imagized text* representation allows us to apply techniques from computer

vision (such as convolutional encoders and image diffusion models) to language. Beyond novelty, the visual approach offers two key advantages: (i) it is intrinsically *multilingual and writing-system-agnostic*, since any writing system (Latin, Chinese characters, hieroglyphs, etc.) can be fed in as images without requiring linguistic preprocessing or dictionaries; (ii) it enables a form of *factorized embedding* akin to color channels in images, which we exploit to disentangle different aspects of linguistic information.

Our Imagized Language Model (ILM) introduces a factorized discrete embedding space or *product codebook* for tokens. Instead of a single vector per word, the model uses multiple codebooks (e.g. multiple “color channels”) to represent each token. This reflects the intuition that a word’s significance arises from a superposition of underlying factors: basic meaning components, syntactic roles, morphological or tonal features, etc. By giving the model separate codebook axes, we encourage it to allocate different facets of language to different sub-codes. For instance, one codebook might encode coarse semantic category, another encodes grammatical function, and another encodes sentiment or tone. In principle, if we had 100 codebooks for meaning and 100 for grammar (as a thought experiment), any sentence could be composed by combining a small set of basis elements from each category. Our implementation uses a smaller number (e.g. 3 codebooks) and lets them be learned automatically, but we will show that appropriate training losses indeed cause these codebooks to align with meaningful latent factors.

To train the codebooks and ensure they capture lexical meaning, we employ *contrastive learning* on pairs of related texts. In supervised settings, these could be question–answer (Q/A) pairs or parallel sentences; in unsupervised settings, we can use automatically generated pseudo-pairs (e.g. consecutive sentences, or masked sentences paired with their contexts). The contrastive objective (InfoNCE) trains the model to align the vector repre-

sentations of related texts while pushing apart unrelated ones: this forces the discrete codes of words with similar meaning or roles to converge, as that is necessary to make the representations of larger texts align. We introduce a secondary loss based on aligning *Gram matrices* of token representations between paired texts. This novel loss encourages the internal *structure* of each sentence’s embedding (the pattern of similarities among the tokens within it) to match between a question and its answer. Intuitively, if a question and answer describe the same scenario or fact in different words, the network must learn consistent relational geometry—for example, a question word like “who” corresponds to a person name in the answer; our Gram alignment loss will pressure the model to embed “who” and that name in a compatible way (e.g. along a shared latent axis for “person”). This helps bridge lexical gaps and induces a form of abstract understanding beyond surface word co-occurrence.

Finally, the ILM incorporates a *diffusion-based decoder* for text generation. Treating a sequence of token embeddings as an image grid of code vectors, we train a 2D U-Net to infill or denoise masked tokens in this grid. This can be seen as a non-autoregressive language model: given a partially corrupted sentence, the model learns to restore missing words using surrounding context, analogous to image inpainting. The process draws inspiration from diffusion probabilistic models [5] that generate data by iterative denoising. In our case, we use a simplified single-step masking diffusion (which can be extended to multi-step iterative refinement): the model learns a conditional distribution of token codes, given their neighbors, that gradually transforms random initial codes into coherent text. This approach allows fully parallel text generation and leverages the rich context captured by our discrete codes.

The core contribution of this work is a unified framework that combines these components to learn language structure **without explicit linguistic supervision**. We provide a detailed mathematical derivation of the training objectives and an intuitive explanation of how they interact to produce meaningful internal representations. We also discuss how this pipeline can be applied to *undocumented languages or scripts* (such as Oracle Bone script, Tangut (Xixia), or Khitan (Qidan) writing) for which no bilingual dictionary exists. By feeding only raw text in the unknown script (rendered as glyph images), the ILM can learn an internal encoding of that language’s words and sentences, essentially constructing its own lexicon and grammar from scratch. This aligns with the distributional hypothesis in linguistics (“you shall know a word by the company it keeps”): the model discovers meaning through patterns of co-occurrence and

context.

In summary, the Imagized Language Model offers a novel way to merge computer vision techniques with language modeling. We demonstrate theoretically how factorized discrete embeddings and contrastive objectives yield emergent semantic structure. In the long term, this approach suggests a path toward machines that learn to **understand new languages solely by reading them**, opening possibilities in historical linguistics, cross-lingual NLP, and resource-poor language modeling.

2 Related Work

Our work intersects multiple research areas:

Contrastive representation learning for language. Contrastive loss functions such as InfoNCE [1] have proven effective in unsupervised representation learning, starting with methods like CPC (Contrastive Predictive Coding) and SimCLR for images. In NLP, they underlie approaches for sentence embedding and alignment across languages. For example, [1] used an InfoNCE objective to learn word representations by predicting future tokens, and others have used contrastive objectives to align bilingual embeddings without supervision. Our use of contrastive learning to align question–answer pairs is related to works on aligning representations of query and response for QA or dialogue tasks, which aim to bridge the lexical gap between questions and answers.

Discrete and factorized embeddings. Representing data with discrete latent codes has a rich history. Vector Quantized-VAE (VQ-VAE) introduced a way to learn discrete codebooks for image and speech representations [2], and later works extended this to product codebooks for higher capacity. Product quantization in embeddings was used classically in nearest neighbor search and more recently in neural networks to get compact yet expressive representations. [2] proposed the Gumbel-Softmax trick to enable differentiable sampling of discrete variables, which we employ to train our codebooks end-to-end. Factorizing an embedding into multiple codes bears resemblance to disentangled representation learning, where different latent dimensions capture independent factors of variation. Our independence loss explicitly encourages each codebook channel to carry information independent of the others, pushing the model toward a disentangled, multi-aspect representation of words. Some recent multilingual models incorporate character glyph information into language modeling, confirming that visual features of text can augment NLP models; our approach goes further by using

glyphs as the sole input and by quantizing the representation into a structured memory.

Diffusion models for text. Diffusion probabilistic models [5] have emerged as a powerful generative framework, primarily in continuous domains like images and audio. Adapting diffusion to discrete data (like text) can be challenging due to non-differentiability of discrete corruption. Recent works have proposed discrete diffusion processes for sequences, including masked token modeling as a special case (sometimes called “mask-based diffusion” or “absorbing state diffusion” for text). [5] introduced a discrete denoising diffusion model for text (D3PM), and subsequent research showed that diffusion-based language models can achieve competitive generation quality while allowing parallel decoding. Our approach fits into this paradigm but operates on a latent space: instead of diffusing over one-hot word tokens directly, we diffuse over continuous glyph feature maps or discrete code embeddings arranged in an image-like grid. This is analogous to latent diffusion in vision models, where the diffusion process is applied in a learned latent space to improve efficiency and semantic coherence. In our case, the latent space is the space of glyph-derived code vectors, which is continuous during training and can be made discrete at generation time. By using a U-Net over a 2D token grid with masks, we effectively learn a conditional masked language model that fills in missing parts of a sentence based on context, akin to the idea of denoising autoencoding used in BERT (although BERT does not use an iterative refinement process).

Unsupervised learning of language structure. Our ultimate goal is to enable learning a language without any bilingual signal (dictionaries or translations), purely from monolingual data of that language. This touches on problems of decipherment and emergent communication. Classical approaches to decipherment often used expectation-maximization to map an unknown script to a known language, requiring assumptions like similar distribution or simple substitution cipher. More recent approaches use neural models to align embedding spaces of languages without supervision, often by exploiting distributional similarity and adversarial training [3] or by initializing from identical patterns (e.g., numerals, names). [3] demonstrated that with sufficient corpus data, one can align word embeddings of two languages through adversarial learning and a shared seed lexicon, effectively learning a bilingual dictionary without direct supervision. Our scenario is even more challenging: we do not assume a second language at all, only a single-language corpus. We rely on the distributional hypothesis and internal consistency

(such as Q/A pairs within the language, or context windows treated as pseudo-translation pairs) to induce meaning. There has been progress in using neural networks to cluster and interpret ancient scripts purely from their internal evidence; for example, [4] used deep clustering on character images and co-occurrence contexts to group symbols of undeciphered writing. Their results indicate that even without understanding the language, one can often recover the inventory of characters and some semantic distinctions by statistical patterns. Our ILM framework builds on similar principles but provides a more powerful end-to-end model that goes from character pixels to an emergent lexicon and language model.

3 Imagized Language Model: Architecture and Objectives

In this section, we present the ILM design in detail. The system consists of four stages or components that are trained in sequence (and partially jointly) to achieve the final language model:

1. **Glyph Encoding:** each token (word or character) is converted into a fixed-size image, and a convolutional neural network (CNN) extracts a glyph feature vector from it (Section 3.1).
2. **Product Code Embedding:** the glyph feature is then passed through a differentiable *product quantization* layer that assigns it to a tuple of discrete code indices (one per codebook channel), producing a compressive discrete embedding (Section 3.2). This forms a kind of *memory table* for word embeddings distributed across multiple codebooks.
3. **Semantic Alignment (QA Training):** using question-answer pairs or related sentence pairs, we fine-tune the code embeddings so that semantically related tokens and sentences map to nearby representations. We use a contrastive InfoNCE loss between questions and answers, and a Gram matrix alignment loss to enforce structural consistency, along with regularizers to keep the code usage broad (Section 3.3).
4. **Diffusion-Based Generation:** we arrange sequences of token codes into 2D grids (simulating text lines or paragraphs as small images) and train a diffusion/inpainting model (a U-Net) that learns to reconstruct masked-out tokens from the surrounding context (Section 3.4). At inference, this model can generate new text by iterative denoising.

Next, we detail each component and its learning objective.

3.1 Glyph Image Encoding

The input to our model is raw text in any language or script. We first convert each token (which could be a word, a character, or a subword unit depending on the language) into an image. For languages with alphabetic scripts like English, we render the word in a fixed-size image (e.g. 128×128 pixels) using a standard font, possibly with an additional encoding of each character’s position (such as a simple grid or one-hot rasterization of each letter). For logographic languages like Chinese, Japanese Kanji, or historical scripts, we can render each character as a 128×128 image using a font or bitmapped glyph. The key is that *the model does not see any numeric token identifiers or one-hot encodings of tokens—it only sees pixels representing the visual appearance of text*. This makes our method inherently applicable to any writing system for which we have images of symbols, without any need for prior linguistic knowledge.

Formally, let x be a token (e.g. the string “cat” or a Chinese character). We define a function $I(x) \in \mathbb{R}^{H \times W \times 3}$ that produces an $H \times W$ RGB image of the token. In our implementation, $H = W = 128$. We then use a convolutional encoder f_{glyph} to obtain a vector representation:

$$g = f_{\text{glyph}}(I(x)) \in \mathbb{R}^{d_g},$$

where $d_g = 128$. The CNN is a lightweight model with a few convolutional layers that reduce the image to a single vector. It is important that f_{glyph} be powerful enough to capture fine distinctions in shape (so that different characters or words map to different features) while also being smooth enough that similar-looking glyphs have similar features. In early training (Stage 1 below), f_{glyph} learns purely from visual data, so it will cluster glyphs by visual similarity, grouping those with common strokes or sub-components. This is desirable as a first step towards grouping by meaning, since often in logographic scripts similar radicals imply related meaning or pronunciation. However, visual similarity alone is not sufficient for semantic grouping, so later stages adjust f_{glyph} and the codes to better reflect usage-based similarity.

3.2 Differentiable Product Code Embedding

After obtaining the glyph feature g , the next step is to map it to a discrete embedding in a **product codebook**. We introduce C separate codebooks (channels), each containing K learnable code vectors of dimension d_e (we usually set $d_e = d_g$). Let $E^{(c)} = \{e_1^{(c)}, e_2^{(c)}, \dots, e_K^{(c)}\}$ denote the set of code vectors in the c -th codebook for $c = 1, \dots, C$. In our experiments we use $C = 3$ and $K = 32$ for each,

meaning the embedding for any token is a combination of 3 code vectors (one selected out of 32 in each channel). The full codebook can be thought of as a K^C discrete grid of possible combinations, akin to a multi-channel color space.

For each channel c , we produce logits $\ell^{(c)} = W^{(c)}g + b^{(c)}$, where $W^{(c)} \in \mathbb{R}^{K \times d_g}$ and $b^{(c)} \in \mathbb{R}^K$. A relaxed categorical choice is made via Gumbel–Softmax [2]. During training we use the straight-through estimator: in the forward pass we take the hard argmax to select a code index k_c , while the backward pass uses the soft sample to allow gradients to flow. The token embedding is the sum of selected code vectors:

$$e = \sum_{c=1}^C e_{k_c}^{(c)}.$$

Summation keeps the output dimension fixed and allows each channel to contribute an additive factor capturing a latent aspect of the token.

3.2.1 Stage 1 Loss: Visual Self-Compression

During Stage 1 we train f_{glyph} and the codebooks purely on glyph images, ensuring that e preserves information in g . We use an InfoNCE objective between glyph features and their codes:

$$\mathcal{L}_{\text{NCE-img}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\langle \hat{g}_i, \hat{e}_i \rangle / \tau_c)}{\sum_{j=1}^B \exp(\langle \hat{g}_i, \hat{e}_j \rangle / \tau_c)},$$

forcing codes to be discriminative for each glyph. Here hats denote ℓ_2 normalization and τ_c is a temperature.

To avoid code collapse we add a usage KL term that matches the empirical usage to a uniform distribution u :

$$\mathcal{L}_{\text{usage}} = \frac{1}{C} \sum_{c=1}^C \text{KL}(p^{(c)} \| u).$$

An independence penalty encourages statistical independence across channels:

$$\mathcal{L}_{\text{indep}} = \frac{2}{C(C-1)} \sum_{a < b} \|\text{Cov}(Y^{(a)}, Y^{(b)})\|_F^2,$$

where $Y^{(c)}$ collects one-hot assignments in channel c . The Stage 1 objective is

$$\mathcal{L}_{\text{code1}} = \mathcal{L}_{\text{NCE-img}} + \lambda_{\text{usage}} \mathcal{L}_{\text{usage}} + \lambda_{\text{indep}} \mathcal{L}_{\text{indep}}. \quad (1)$$

Optimizing this yields a visual lexicon where glyphs with similar strokes share portions of their codes.

3.3 Semantic Alignment via QA Training

Stage 2 introduces semantic signals. We assume access to paired texts (true Q/A pairs or pseudo-pairs derived from context). For each question-answer pair (Q, A) we compute sentence embeddings by averaging token codes: $h_Q = \frac{1}{|Q|} \sum_{x \in Q} e_x$, $h_A = \frac{1}{|A|} \sum_{x \in A} e_x$, followed by normalization. A bidirectional InfoNCE loss brings true pairs together:

$$\mathcal{L}_{\text{NCE-QA}} = -\frac{1}{B} \sum_{b=1}^B \left[\log \frac{\exp(\langle h_{Q_b}, h_{A_b} \rangle / \tau_{\text{qa}})}{\sum_{j=1}^B \exp(\langle h_{Q_b}, h_{A_j} \rangle / \tau_{\text{qa}})} + \log \frac{\exp(\langle h_{Q_b}, h_{A_b} \rangle / \tau_{\text{qa}})}{\sum_{j=1}^B \exp(\langle h_{Q_j}, h_{A_b} \rangle / \tau_{\text{qa}})} \right] \quad (2)$$

We also align internal structure via Gram matrices. Let Z_Q and Z_A be row-stacked, normalized token embeddings for Q and A . Define $G_Q = Z_Q Z_Q^\top$ and $G_A = Z_A Z_A^\top$. After padding to a common size we minimize

$$\mathcal{L}_{\text{Gram}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{m_b^2} \|G_Q^{(b)} - \Pi G_A^{(b)} \Pi^\top\|_F^2, \quad (3)$$

where m_b is the question length and Π handles padding. This pressures the relational geometry of tokens in a question to match that of its answer, encouraging alignment of semantic roles.

Regularizers $\mathcal{L}_{\text{usage}}$ and $\mathcal{L}_{\text{indep}}$ remain active. The Stage 2 objective becomes

$$\mathcal{L}_{\text{code2}} = \mathcal{L}_{\text{NCE-QA}} + \lambda_{\text{Gram}} \mathcal{L}_{\text{Gram}} + \lambda'_{\text{usage}} \mathcal{L}_{\text{usage}} + \lambda'_{\text{indep}} \mathcal{L}_{\text{indep}}, \quad (4)$$

The combination of sentence-level and structure-level alignment yields emergent semantics: synonyms cluster, question words align with their answer types, and channels specialize. The codebooks now encode not only how a glyph looks but also how it behaves in language.

3.4 Diffusion-Based Text Infilling

Stage 3 trains a diffusion-style denoiser over sequences of token embeddings. We arrange a sentence's embeddings into an $H \times W$ grid F . A binary mask M with mask ratio β selects which positions to corrupt; we set those embeddings to zero to obtain $\tilde{F}$. A U-Net U receives $(\tilde{F}, M)$ and outputs reconstructions $\hat{F}$. The loss is a masked MSE:

$$\mathcal{L}_{\text{diff}} = \frac{1}{\sum_{i,j} M_{i,j}} \sum_{i,j: M_{i,j}=1} \|\hat{F}_{i,j} - F_{i,j}\|_2^2. \quad (5)$$

This teaches the model to inpaint missing embeddings using their visual/semantic neighbors. At inference we can start from a fully masked grid and iteratively (or in one shot) predict embeddings, then

map them back to tokens by finding the nearest code combination. Because the embeddings already encode semantics, the denoiser leverages high-level context rather than surface n-grams, enabling coherent generation even in languages with complex morphology or free word order.

3.5 Why Unsupervised Semantic Learning Emerges

Combining Stages 1–3 yields the overall objective

$$\mathcal{L} = \mathcal{L}_{\text{code1}} + \mathcal{L}_{\text{code2}} + \lambda_{\text{diff}} \mathcal{L}_{\text{diff}}, \quad (6)$$

with λ_{diff} controlling the influence of the diffusion term. The InfoNCE losses maximize mutual information between glyphs and codes, and between related texts. The Gram loss enforces relational consistency. The diffusion loss requires that embeddings remain predictive of local context. Together they leave little room for trivial solutions: the code must encode everything needed to reconstruct glyph appearance, align semantic roles, and support context-based prediction. This multiview pressure naturally disentangles factors of meaning across codebook channels.

4 Applying ILM to Undocumented Scripts

The ILM pipeline is well suited to languages lacking annotations or dictionaries.

Glyph collection. Gather images for each distinct symbol. For inscriptions, segment characters manually or with vision tools. Normalize to a standard canvas size.

Stage 1 (visual clustering). Train f_{glyph} and the codebooks using raw glyph images. This recovers variants and radicals, forming an initial inventory.

Stage 2 (pseudo-semantic alignment). Construct pseudo-pairs from the corpus: consecutive sentences, masked spans, or topical groupings. Train with $\mathcal{L}_{\text{code2}}$. Even noisy pairs cause distributional clustering (words used in similar contexts converge). Channels may capture categories such as names, locations, or actions purely from usage patterns.

Stage 3 (diffusion language model). Train the infiller on raw sequences. The model learns to restore tokens consistent with context, effectively modeling the language. Generated sentences can be inspected for plausibility, giving insight into syntax and collocations.

Evaluation without annotation. Since gold labels are absent, we rely on indirect signals: cluster coherence, reconstruction quality, and expert assessment. Linguists can inspect tokens sharing the same code to hypothesize meaning. The diffusion model’s ability to fill gaps in inscriptions serves as a practical validation.

5 Discussion

ILM works because each component constrains the representation from a different angle:

- Visual grounding retains orthographic distinctions without handcrafted tokenizers.
- Product codes provide a discrete, memory-like table with factored capacity, steering the model toward interpretable latent dimensions.
- Contrastive alignment injects semantic pressure, clustering tokens by usage.
- Gram alignment adds structural supervision, approximating grammatical roles.
- Diffusion infilling ensures embeddings remain useful for generation, validating that learned codes capture enough context to predict missing words.

The framework therefore unifies perception (glyphs) and language structure under self-supervision. When applied to modern languages, ILM can support tasks like QA, summarization, and translation through the diffusion decoder. When applied to ancient scripts, it provides a new tool for decipherment by revealing latent categories and usage patterns.

6 Conclusion

We presented the Imagized Language Model, a factorized discrete embedding architecture trained solely from glyph images and self-supervised objectives. By combining product quantization, contrastive sentence alignment, Gram-structure regularization, and diffusion-based infilling, the model learns semantically meaningful representations without any linguistic supervision. The learned codebooks behave like a structured lexicon, opening the door to learning new languages purely by reading their texts. Future work will extend ILM to multi-step diffusion decoding, joint multilingual training, and large-scale experiments on historical corpora.

References

- [1] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- [2] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations (ICLR)*, 2017.
- [3] Alexis Conneau, Guillaume Lample, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Word translation without parallel data. In *International Conference on Learning Representations (ICLR)*, 2018.
- [4] Logan Born, M. Willis Monroe, Kathryn Kelley, and Anoop Sarkar. Learning the character inventories of undeciphered scripts using unsupervised deep clustering. In *Proceedings of the Workshop on Computation and Written Language (CAWL)*, 2023.
- [5] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rahul G. Krishnan. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.